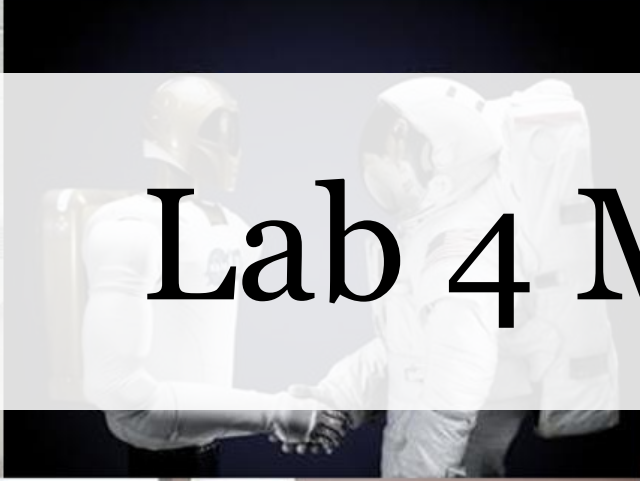
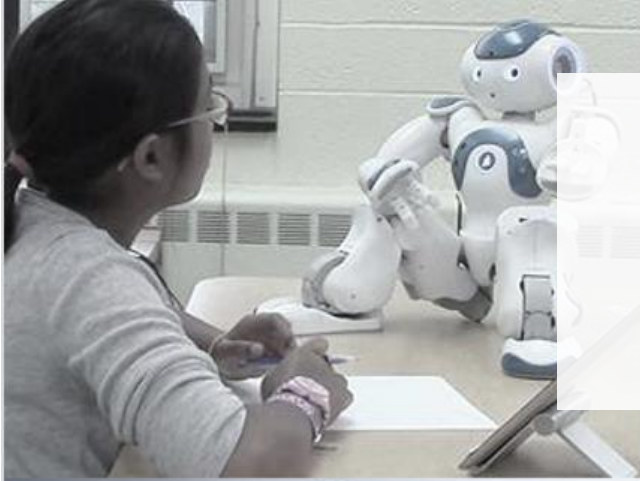
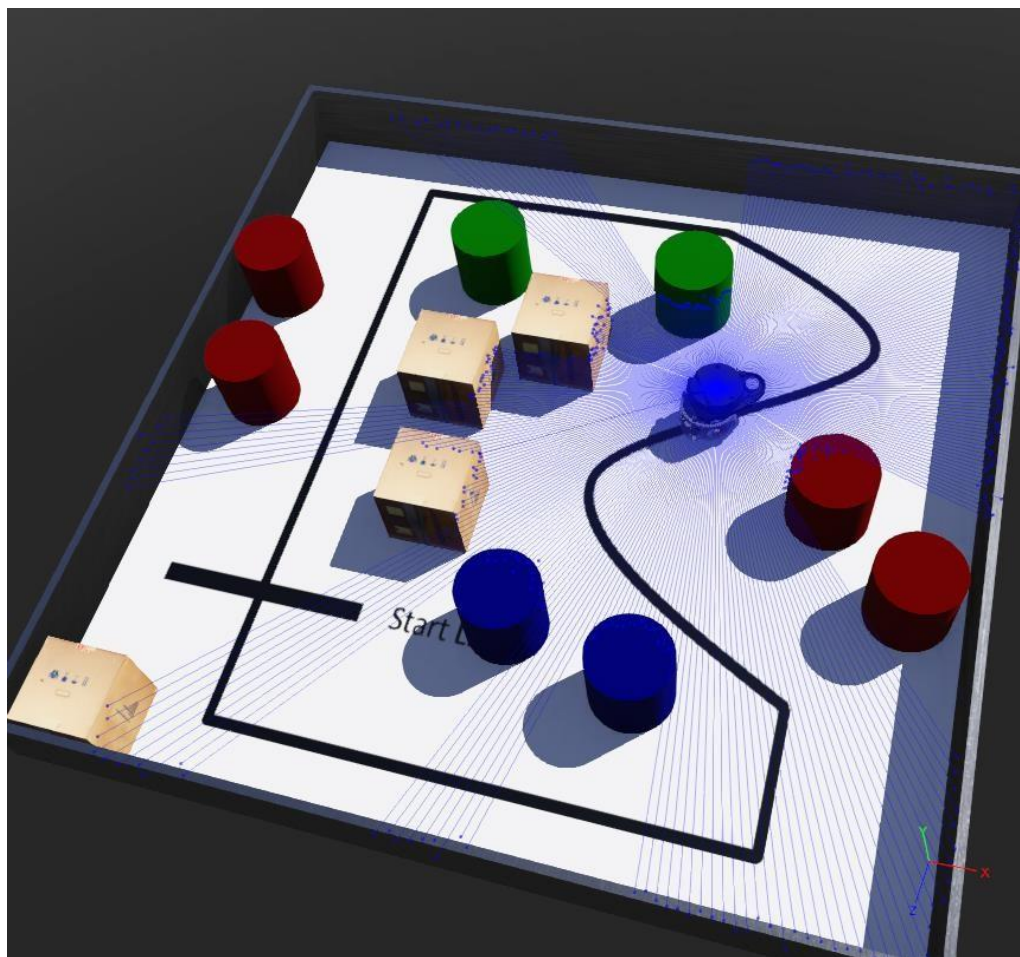






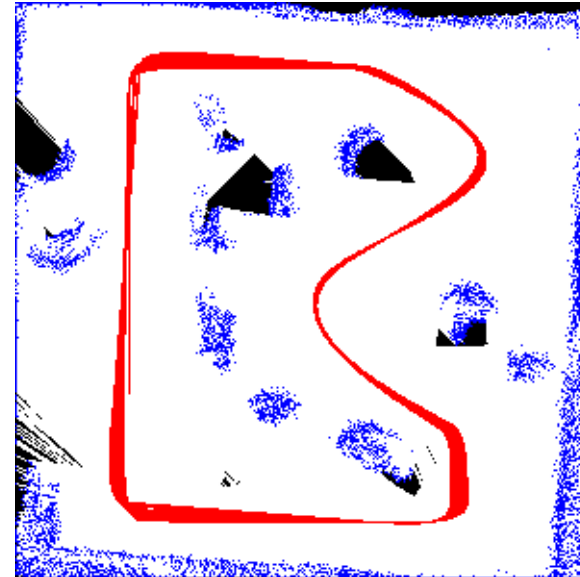
LiDAR



Distance Range	120 ~ 3,500mm
Distance Accuracy (120mm ~ 499mm)	±15mm
Distance Accuracy(500mm ~ 3,500mm)	±5.0%
Distance Precision(120mm ~ 499mm)	±10mm
Distance Precision(500mm ~ 3,500mm)	±3.5%
Scan Rate	300±10 rpm
Angular Range	360°
Angular Resolution	1°

Mapping an Environment

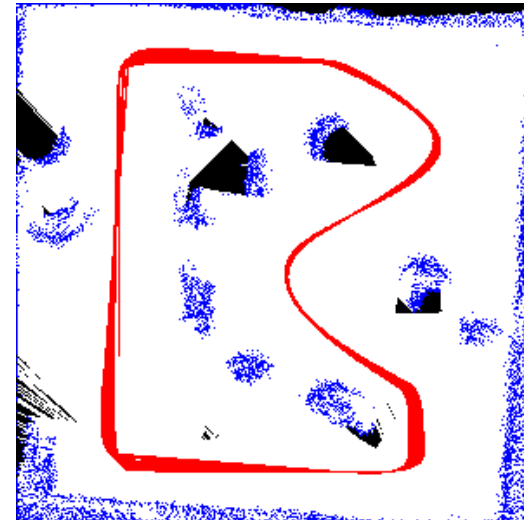
A robot can read the sensor into their memory to create a map.



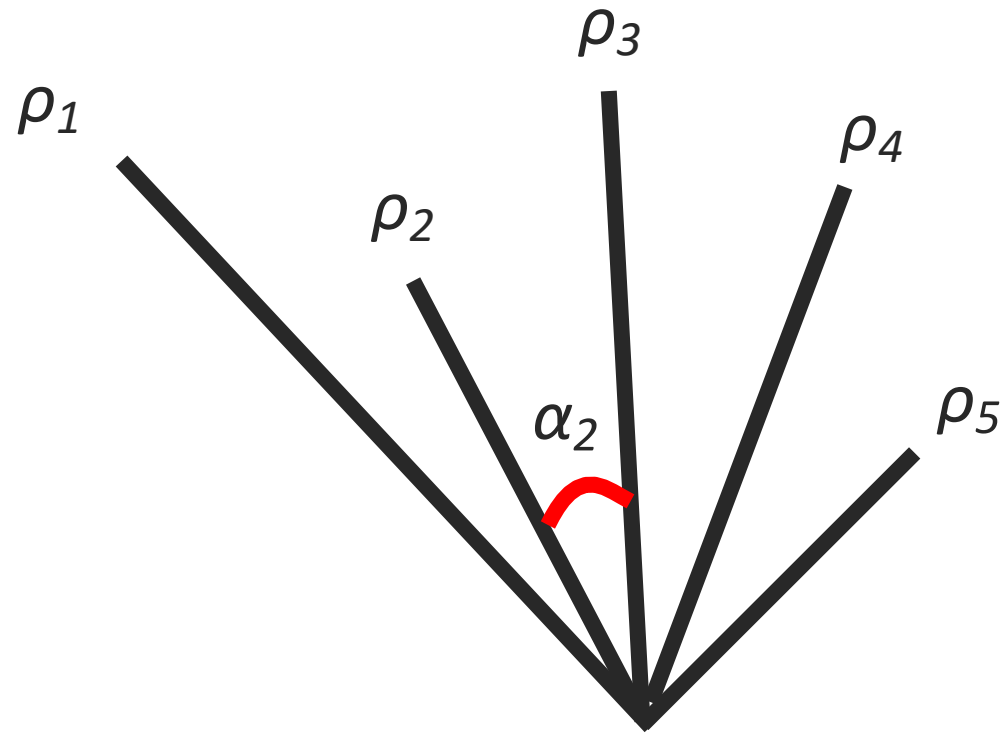
Mapping an Environment

To map an environment, the robot requires:

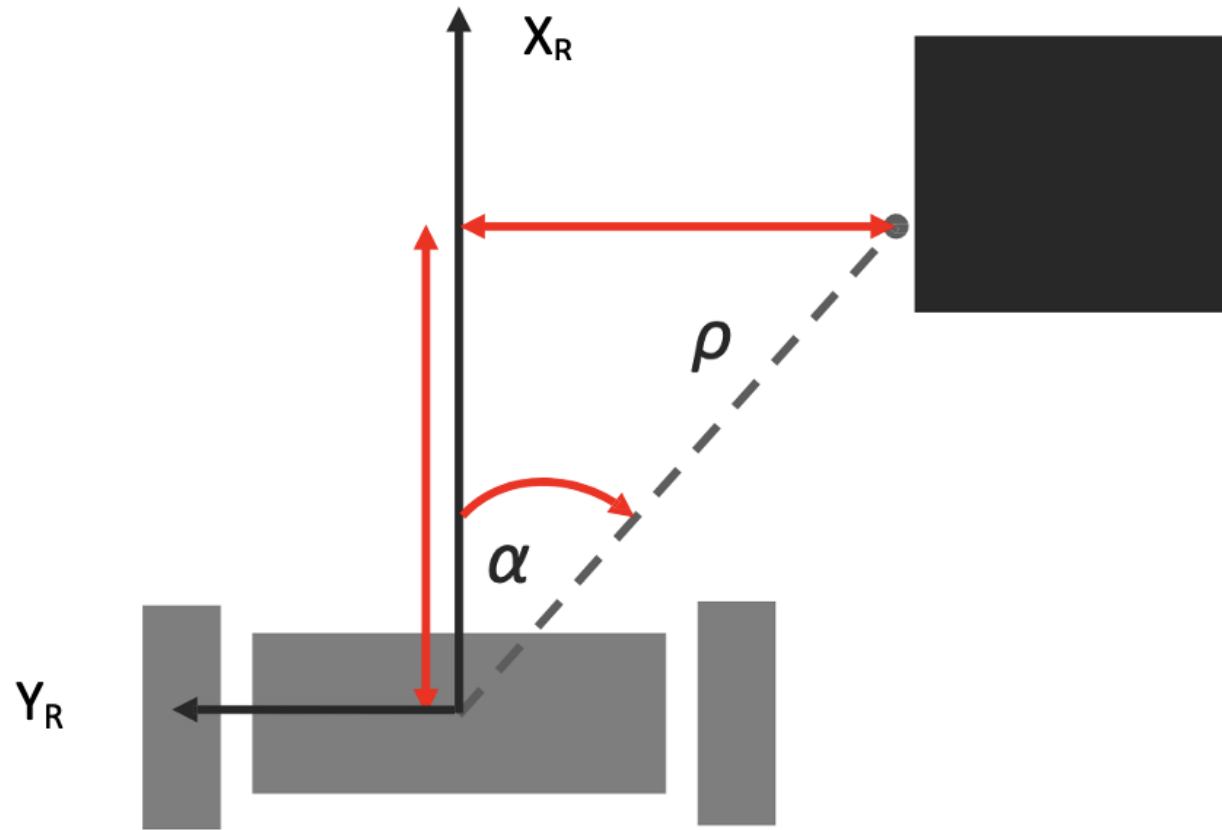
- **Local Sensing:** LiDAR identifies obstacle positions relative to the robot.
- **Global Positioning:** Transform locations from the robot's frame to the map frame.
- **Discrete Coordinate Mapping:** Convert continuous coordinates to discrete grid locations on the display.



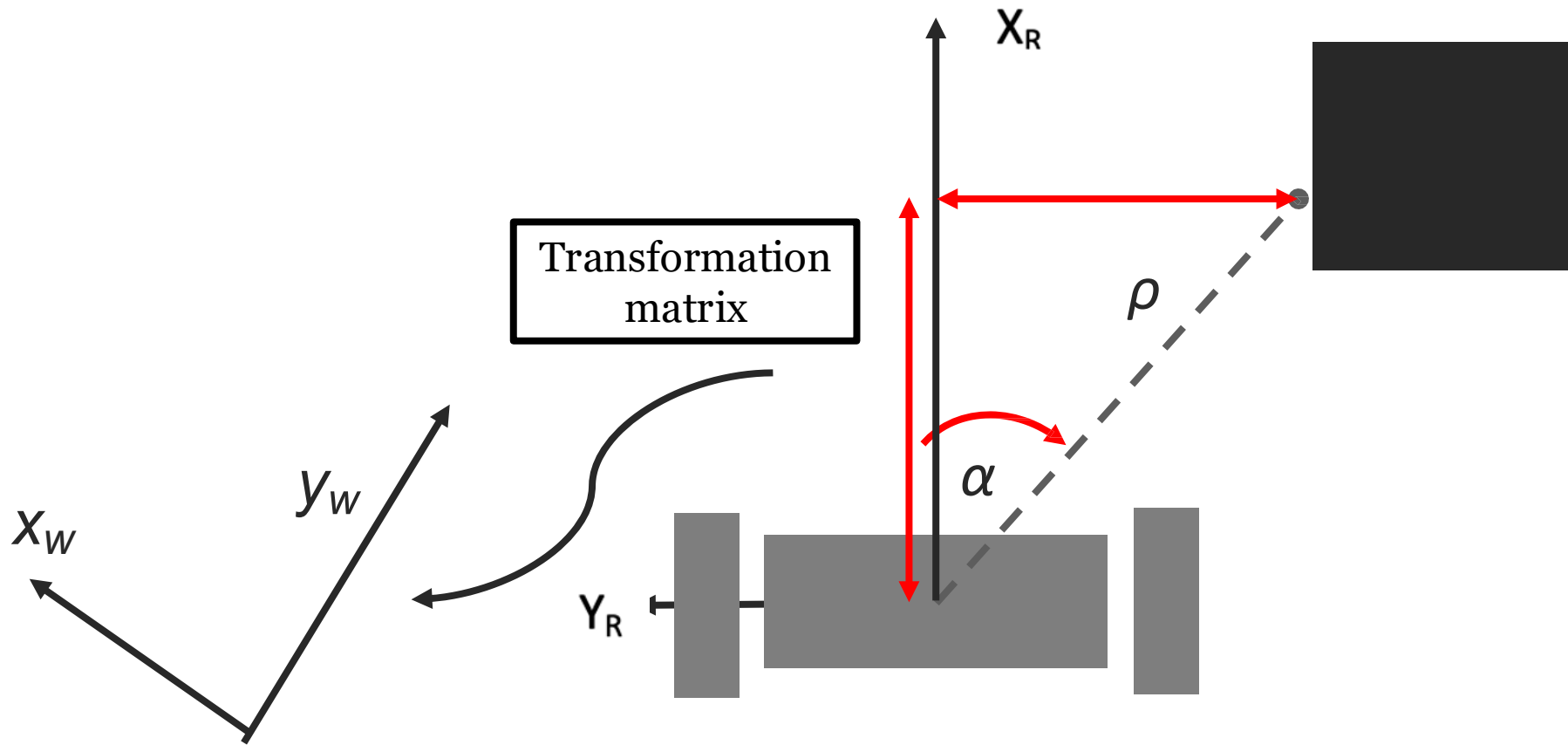
LiDAR rays are discrete and at uniform interval

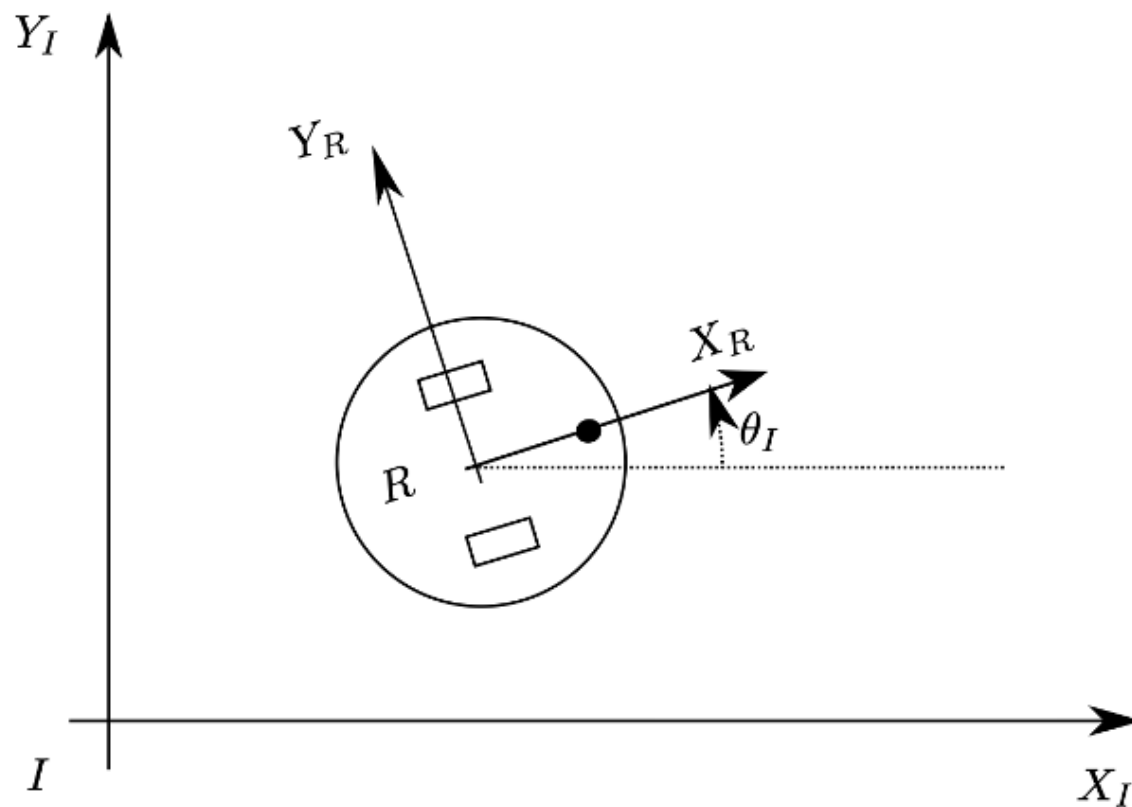


Convert the sensor readings into the robot coordinates



Transform the robot coordinates into the map coordinates





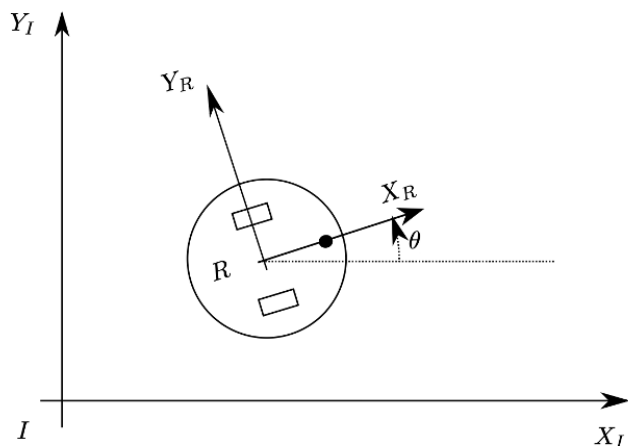
Solution

The robot frame is rotated by an angle θ relative to the global frame (about the out-of-plane Z axis), then the 2D rotation that maps coordinates from $\{R\}$ to $\{I\}$ is:

$${}^I R_R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

The map coordinate system has its origin (0,0) at the top-left corner of the arena and its x-axis increases to the right, and its y-axis increases downward.

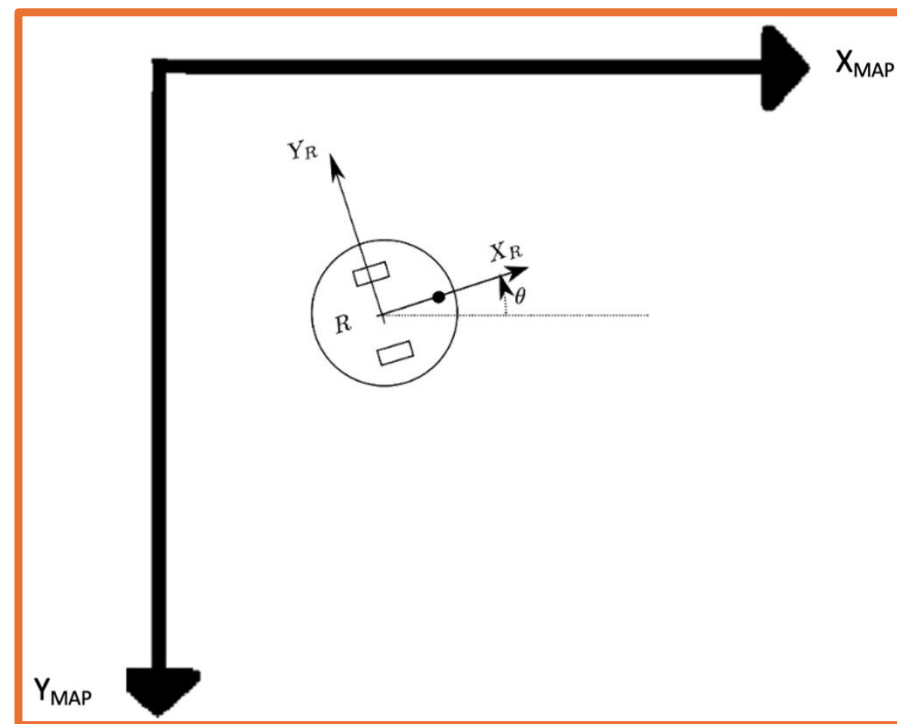
On the other hand, in our defined robot frame, forward motion aligns with the +x-axis, and left corresponds to the +y-axis.



Solution

The robot frame is rotated by an angle θ relative to the global frame (about the out-of-plane Z axis), then the 2D rotation that maps coordinates from $\{R\}$ to $\{I\}$ is:

$${}^I R_R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$



The goal frame changes. What is the transformation matrix now?

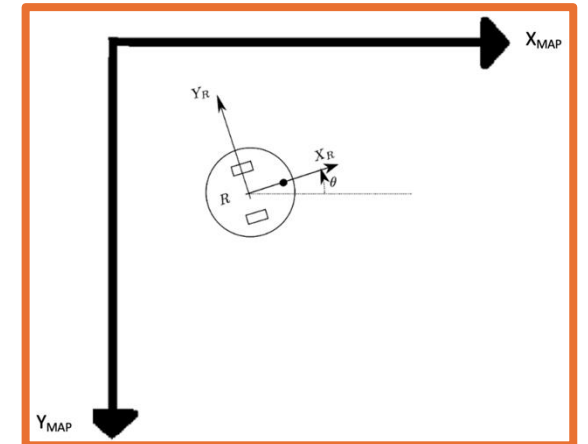
Recall: Mapping from one frame to another

The transformation matrix from the robot's frame $\{X_B, Y_B\}$ to the global reference frame $\{X_A, Y_A\}$:

$${}^A R_B = \begin{bmatrix} \hat{x}_B \cdot \hat{x}_A & \hat{y}_B \cdot \hat{x}_A \\ \hat{x}_B \cdot \hat{y}_A & \hat{y}_B \cdot \hat{y}_A \end{bmatrix}.$$

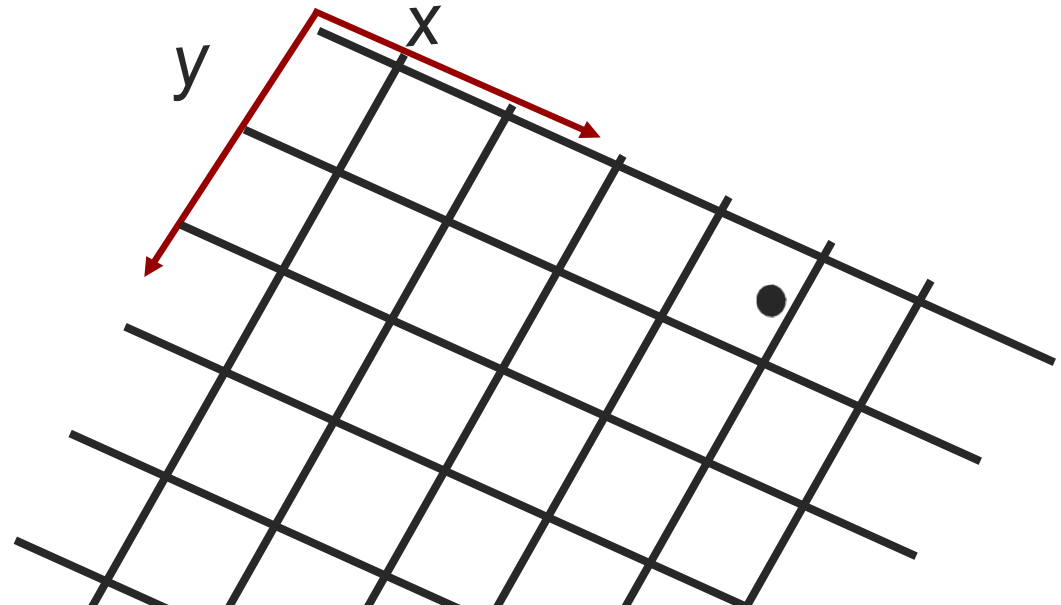
With the position of the robot in the reference frame, we can use the homogeneous transformation matrix to convert the relative location into the map coordinates.

$$\begin{bmatrix} {}^A Q \\ 1 \end{bmatrix} = \begin{bmatrix} {}^A R & | & {}^A P \\ \hline 0 & 0 & 0 & | & 1 \end{bmatrix} \begin{bmatrix} {}^B Q \\ 1 \end{bmatrix}$$



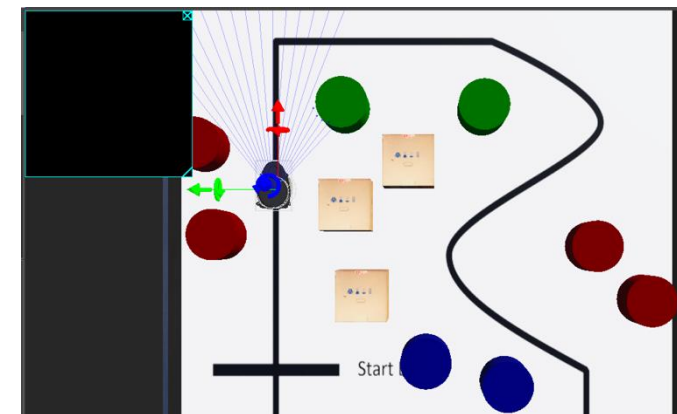
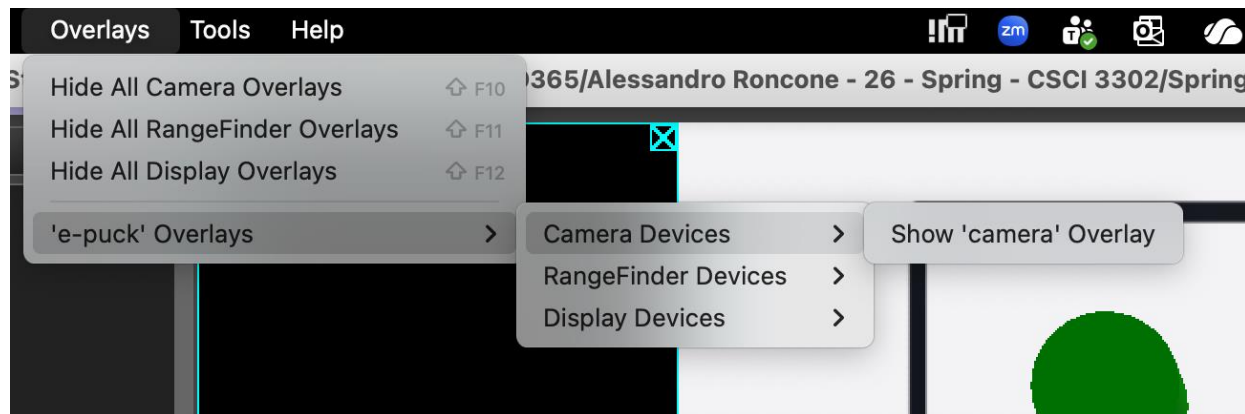
Convert the continuous map coordinates into the discrete display pixel locations

- Poses are continuous coordinates, but the display uses discrete pixels. How do we convert continuous coordinates to discrete pixels?
- Lab 4 Environment:
 - The display has 300 x 300 pixels.
 - Robot moves on a 1 m $\times$ 1 m floor with x and y coordinates ranging approximately from 0 to 1.



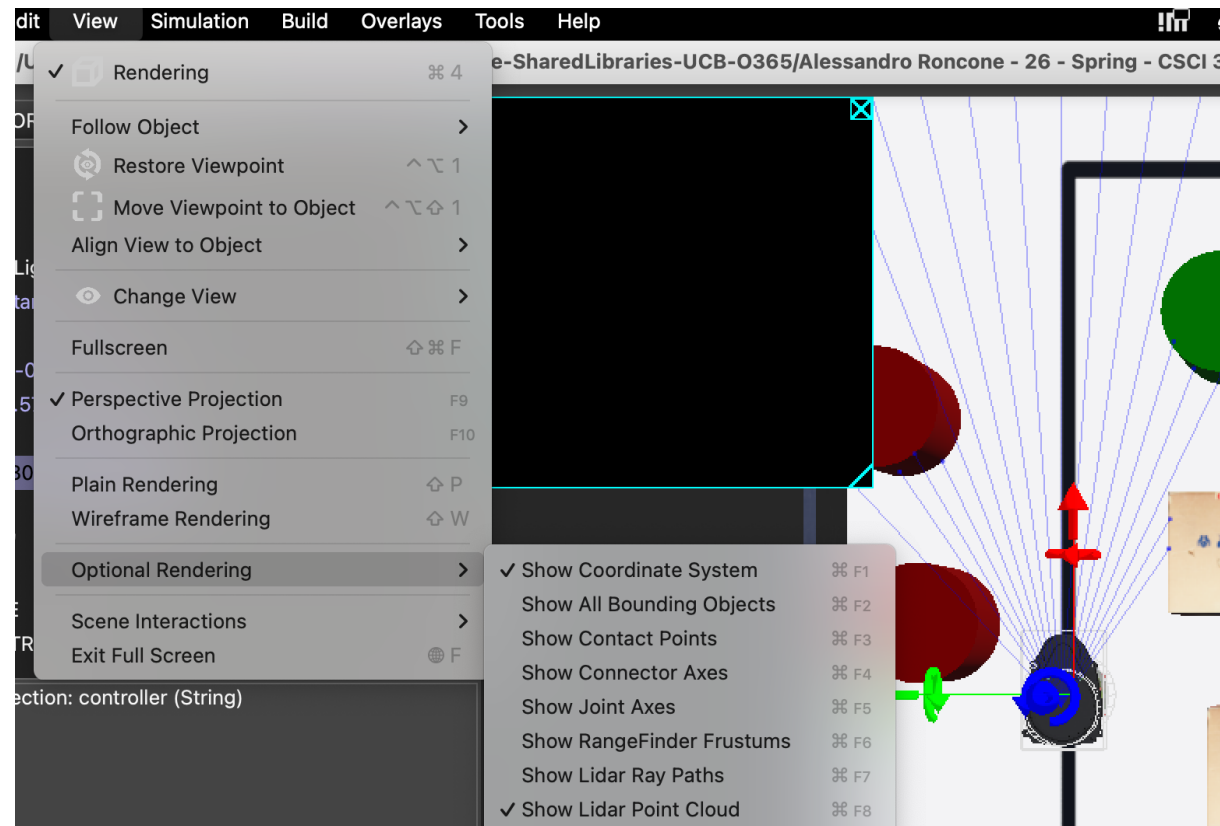
Notes

- This is a 1-week lab. **Due on the coming Tuesday, March 3rd.**
- If you accidentally turn off the 'display' overlay(**the black rectangle surrounded by blue line**), please click **Show 'camera' Overlay** after choosing **Overlays** in the top menu.



Notes

- If you cannot see multiple blue rays from the e-puck robot, you can enable it through: **View > Optional Rendering > Show Lidar Point Cloud**



Notes

- Python package install for Webots.
 - If *pip install package_name* doesn't work in Webots, follow this process to target the correct Python environment.
 - Find Webots' Python path:
 - Mac OS: Webots > Preferences > General > Python
 - Windows: Tools > Preferences > General > Python
 - Example path: */opt/anaconda3/envs/webots_lab4/bin/python*
 - Install with matching pip:
 - Example of installing numpy: */opt/anaconda3/envs/webots_lab4/bin/pip install numpy*